

hsh

Product & Engineering Roadmap

Canonical milestone sequence aligned to SRS v0.2 and UML 2 Component Design v0.2

Field	Value
Document version	0.2
Product	hsh
Architecture	GNU Bash 5.3.x / C core + per-session Rust agent
MVP milestone	M1 - Explicit AI + read-only execution
Milestone system	M0-M6 canonical sequence
Status	Implementation roadmap baseline
Date	11 September 2026

Normative sequencing rule
No milestone may introduce a capability whose prerequisite safety/context gate is scheduled later. In particular, mutation requires M2 policy/authorization; destructive follow-up requires M3 typed context; automatic NL routing waits until M5.

1. Purpose and Alignment

This roadmap replaces v0.1 and defines one canonical implementation sequence for hsh. It resolves the milestone drift between the previous roadmap, SRS and component design by treating the SRS v0.2 requirements and UML v0.2 execution boundary as fixed architectural constraints.

1.1 Non-negotiable architecture constraints

- Bash Compatibility Core in C is the only canonical OS process/pipeline/job-control executor.
- The Rust agent plans, derives capabilities, applies policy, manages context/provider integration and issues authorization artifacts; it does not own an alternate exec path.
- ExecutionPlan and IPC envelopes are versioned from M1, not retrofitted later.
- Provider-supplied risk/capability labels are hints only; effective capabilities are recomputed locally from typed steps, cwd, env and targets.
- LLM output is never passed to eval, bash -c or an unrestricted shell-script step.
- Context output is data, not instruction; provider egress is allowlisted and raw stdout/file content is denied by default.
- One hsh-agent child is owned by one interactive hsh session for M1-M5; no shared user daemon is required by this roadmap.

1.2 Target evolution

M0 foundation

- > M1 explicit AI + read-only vertical slice
- > M2 controlled mutation + confirmation/authorization
- > M3 bounded typed context + stale-target protection
- > M4 multi-provider/config maturity
- > M5 opt-in automatic NL routing
- > M6 hardening, packaging and 1.0

2. Canonical Milestone Table

ID	Milestone	Primary exit	Target	Assist tokens*
M0	Foundation & Provenance	Pin Bash baseline; build/repo/licensing baseline; protocol decisions	0.0.0-dev	20k-40k
M1	MVP - Explicit AI / Read-only	`ai` vertical slice; one provider; typed plan; minimal versioned Policy Engine; canonical Bash executor	0.0.1	70k-110k
M2	Controlled Mutation	Typed write/delete/process/privilege operations; confirmation; authorization artifact; environment-aware policy	0.0.2	60k-100k
M3	Bounded Context	Typed ContextResult; TTL/job scope; identity revalidation; safe destructive follow-up	0.0.3	50k-90k
M4	Provider & Config Maturity	At least one remote + one local provider; stable adapter/config/egress semantics; failure isolation	0.1.0	45k-80k
M5	Opt-in Automatic NL Routing	Prefixless multilingual routing with Bash precedence; ambiguity contract; explicit force modes	0.2.0	45k-80k

ID	Milestone	Primary exit	Target	Assist tokens*
M6	Hardening & 1.0	Security/fuzzing, platform matrix, packaging, license checks, audit retention/integrity, release soak	1.0.0	100k-170k

*Planning estimate only. See Section 11 for measurement assumptions. M0 is a prerequisite engineering gate; M1 remains the first user-meaningful MVP milestone.

Single numbering rule

Feature plans, pull requests, SRS acceptance criteria and UML component exits should reference only M0-M6 above. No separate “roadmap milestone numbering” is permitted.

3. M0 - Foundation & Provenance

M0 removes dependencies that would otherwise make M1 acceptance subjective. It is not a product release; it is the engineering baseline required before MVP implementation.

3.1 Required decisions / deliverables

Item	Decision / output	Gate
Bash baseline	Pin an exact GNU Bash 5.3.x upstream tag/commit; record provenance and upstream remote policy.	Required
Primary M1 platform	Select one blocking platform: Linux or macOS. The secondary platform is smoke/non-blocking until M6.	Required
Bash smoke suite	Freeze a named subset of upstream tests plus hsh-specific direct-command fixtures.	Required
Agent lifecycle	Per-interactive-session child process; parent owns start/stop/restart epoch and fallback to direct Bash.	Required
Initial provider	Choose one M1 adapter. CI uses a deterministic mock; live-provider tests are opt-in.	Required
Protocol v1	Freeze envelope kinds, schema_version, error taxonomy, plan hash algorithm and version negotiation.	Required
GPL/distribution	Document source/provenance obligations and perform pre-distribution legal review gate.	Required before binary distribution

3.2 M0 exit criteria

M0-AC-01: Clean clone builds the unmodified/pinned Bash baseline using documented prerequisites.

M0-AC-02: Primary platform and secondary smoke platform are recorded.

M0-AC-03: Bash compatibility smoke suite is version-controlled and runnable.

M0-AC-04: IPC v1 and ExecutionPlan v1 schemas contain explicit version fields before executable AI code lands.

M0-AC-05: Repository layout matches SRS/UML ownership boundaries; no Rust OS-executor crate exists.

M0-AC-06: License/provenance checklist exists before any public binary artifact.

4. M1 - MVP: Explicit AI + Read-only Vertical Slice

Goal: deliver a buildable hsh that behaves as Bash for ordinary input and supports an explicit `ai "...` path that can perform useful read-only tasks through the complete C -> Rust -> provider -> typed plan -> local capability derivation -> minimal policy -> canonical Bash execution -> typed result flow.

4.1 MVP scope

- Fork/rebrand the pinned Bash baseline as hsh without automatic NL routing.
- Implement explicit `ai` entry point and session-owned C<->Rust IPC bridge.
- Implement Request/Response/Plan/Result envelopes with schema_version from day one.
- Implement one AIProvider adapter plus deterministic mock provider for CI.
- Implement typed ProcessStep/PipelineStep/Glob/Redirection/InputRef subset required for read-only inspection.
- Introduce Policy Engine v1 as a named component: default-deny, locally derived read-only capabilities, bounded cwd/path rules and no mutation capability.
- Execute approved plans only through Bash Compatibility Core.
- Return stdout/stderr/exit status to the local response layer with output/size/time limits.
- Format MVP summaries locally from typed metadata. Raw stdout/body is not resent to the provider by default.
- Add protocol/parser adversarial tests and schema fuzzing as P1 inside M1, not postponed to public beta.

4.2 Explicitly out of scope

- File writes/deletes, process kill, privilege escalation and package installation.
- Automatic prefixless NL routing.
- Destructive conversational follow-ups.
- Persistent context/memory.
- Multi-provider routing.
- Provider-side summarization of arbitrary command output.
- Shared background agent daemon.
- Raw shell-script or bash -c plan steps.

4.3 M1 work packages and dependencies

WP	Deliverable	Depends on	Estimate
WP1	hsh fork/rebrand + primary-platform reproducible build	M0 Bash baseline	10k-18k
WP2	Bash smoke fixtures + direct-path regression harness	M0 smoke suite	6k-10k
WP3	Protocol v1 codecs + C bridge + session child lifecycle	M0 protocol/lifecycle	12k-20k
WP4	Rust agent skeleton + strict plan parser	WP3	10k-16k
WP5	AIProvider trait + selected M1 adapter + mock	WP4, M0 provider	10k-18k
WP6	ExecutionPlan v1 read-only step subset	WP3/4	8k-14k
WP7	Policy Engine v1 stub: derive + default-deny + read-only allow rules	WP6	8k-14k
WP8	Canonical Bash executor adapter + streaming/result capture	WP3/6/7	10k-18k
WP9	Adversarial/fuzz/error-path tests + docs	WP3-8	10k-18k

4.4 M1 acceptance criteria

M1-AC-01: On the selected primary platform, a clean clone builds hsh reproducibly; the secondary platform is smoke/non-blocking and its status is recorded.

M1-AC-02: `hsh -c "echo ok"` plus the frozen M0 Bash smoke suite pass on the primary platform.

M1-AC-03: Normal Bash commands do not start/contact the AI provider path.

M1-AC-04: `ai "...` obtains a schema-versioned typed plan from the configured provider or deterministic mock.

M1-AC-05: Malformed/unknown-version plans are rejected before authorization or process creation.

M1-AC-06: Policy Engine v1 recomputes effective capabilities; provider risk/capability hints cannot authorize execution.

M1-AC-07: Only read-only capabilities supported by M1 may authorize; write/delete/kill/network mutation/privilege steps are denied as unsupported/default-deny.

M1-AC-08: The sole OS execution path is Bash Compatibility Core; Rust cannot bypass policy with direct exec.

M1-AC-09: No raw LLM text is passed to eval/bash -c/unrestricted shell parsing.

M1-AC-10: Exit status and bounded stdout/stderr are available locally; MVP natural-language-like presentation is generated from approved metadata without a second provider call over raw output.

M1-AC-11: Provider/agent failure leaves the interactive shell usable and direct Bash path available.

M1-AC-12: Protocol/parser tests include malformed JSON, oversized fields, unknown step types, command-smuggling strings and authorization bypass attempts.

M1 policy boundary

M1 does not wait for M2 to gain policy. It ships Policy Engine v1 with a deliberately small read-only rule set. M2 expands the same component to state-changing capabilities; it does not replace an executor-side allowlist hack.

5. M2 - Controlled Mutation

M2 is the first milestone permitted to authorize state-changing AI-mediated actions. It expands the existing Policy Engine and IPC lifecycle rather than adding a parallel mutation path.

- Define typed FileOperationStep, ProcessOperationStep and PrivilegeStep semantics; no free-form selection expression.
- Derive capabilities from steps, canonical targets, cwd, env_delta, redirections and host/filesystem profile.
- Add multi-phase IPC: plan -> ConfirmationChallenge -> ConfirmRequest/CancelRequest -> AuthorizationArtifact -> execute/stream/result.
- Bind authorization to plan_hash, session/restart epoch, immutable config/policy snapshot, uid/gid/cwd and expiry.
- Render preview/confirmation in the C frontend TTY; Enter cancels by default and non-interactive AI mutation is denied unless a future explicit policy mode is designed.
- Revalidate targets immediately before mutation; deny symlink/scope ambiguity according to policy.
- Add timeout/output/subprocess limits to mutation paths and auditable local decisions.

5.1 Exit criteria

M2-AC-01: Write/delete/process-control actions cannot execute without an authorization artifact valid for the exact plan and snapshot.

M2-AC-02: Mutating the plan, cwd/env-derived risk inputs or policy/config snapshot invalidates prior approval.

M2-AC-03: Confirmation UI and cancel semantics work over the versioned IPC state machine.

M2-AC-04: Network and privilege capabilities remain default-deny unless explicitly implemented and policy-enabled.

M2-AC-05: All state-changing steps use the canonical Bash executor path.

6. M3 - Bounded Typed Context

Context precedes automatic routing. M3 makes references such as “them”, “the third one” or “kill it” safe enough to evaluate before M5 allows prefixless NL input.

- Introduce ContextResult with typed target identities, originating job_id, TTL, size bounds and source plan/result hashes.
- Define explicit same-job reference resolution; cross-job pronouns do not implicitly bind.
- Revalidate PID identity/start-time and file identity/path state before destructive follow-up.
- Deleted/recreated or changed targets become stale and require re-resolution; PID reuse cannot target a new process.
- Provider context export uses an allowlist of typed metadata. Prior stdout/file body remains DATA and denied by default.
- Every context-derived mutation undergoes fresh capability derivation, policy evaluation and confirmation.
- Add prompt-injection adversarial fixtures where stdout/file names attempt to alter policy or request secrets.

7. M4 - Provider & Configuration Maturity

- Implement at least one remote provider and one local provider behind the same IAIPProvider contract.

- Formalize TOML user/system/project configuration precedence and credential separation.
- Keep provider-specific structured-output translation outside policy/executor semantics.
- Apply the same Data-Egress Gate and classification rules to every provider adapter.
- Isolate provider failures/timeouts so they never crash the shell or invalidate direct Bash use.
- Use immutable config/policy snapshots per plan authorization; reload applies to subsequent planning/authorization only.
- Add provider conformance tests for protocol normalization and unsupported-model behavior.

8. M5 - Opt-in Automatic Natural-Language Routing

Prefixless NL routing arrives only after policy, authorization and bounded context are established. It is opt-in and must preserve Bash precedence.

- Valid Bash syntax and explicit direct-command modes win over automatic routing.
- Unknown-command fallback must visibly indicate rerouting before planning; high-risk ambiguous input never auto-executes.
- Provide explicit ``ai`` and direct-shell force modes regardless of auto-routing setting.
- Compatibility mode disables auto-routing and ambiguous hsh extensions.
- Route multilingual input without language-specific execution semantics.
- Route diagnostics explain why input was classified as Bash, explicit AI, fallback AI or rejected ambiguity.
- Automatic routing cannot broaden context job scope or bypass M3 reference-resolution rules.

9. M6 - Hardening, Platform Matrix and 1.0

- Run compatibility regression against pinned/upstream Bash fixtures and document intentional deviations.
- Promote secondary desktop/server platform to blocking support; document WSL status and OS adapter limitations.
- Expand fuzzing across JSON/IPC, typed-plan parser, policy inputs and authorization artifacts; perform threat-model review.
- Crash/restart isolation tests prove agent failure cannot corrupt Bash job-control state.
- Finalize audit retention, redaction and integrity controls.
- Package signed/reproducible artifacts where applicable; validate GPL notices/source provenance before distribution.
- Freeze stable v1 provider/protocol contracts and security response/upstream merge policy.
- Run release-candidate soak; close blockers before 1.0 tag.

10. Developer Workflows Are Capabilities, Not a Shortcut Milestone

Repository-aware workflows are layered onto M2-M5 capabilities rather than assigned a standalone milestone that could bypass policy. Git/build/test operations use the same typed plan, capability derivation, authorization and context rules.

Workflow	Required capabilities / rules
git status/diff/log	filesystem.read; process.execute read-only profile
git commit / file formatting	filesystem.write; confirmation/policy when AI-mediated
git push / fetch	network.outbound plus repository/process capabilities; no special bypass
npm/gradle test	process.execute; filesystem.write/network.outbound derived from declared/env-observed step semantics; bounded output/time
hooks/scripts	Treated as spawned subprocess effects; policy profile may deny untrusted repo hooks or network/privilege effects
failure explanation	Captured output is local DATA; provider export requires explicit egress policy/redaction, not automatic raw forwarding

11. Planning Estimates and Measurement Assumptions

Token ranges are capacity-planning heuristics for AI-assisted engineering, not earned-value metrics. A milestone is complete only by acceptance criteria.

Assumption	Definition
What counts	Approximate total input + output tokens used in implementation/review/debugging conversations and coding-agent work for the milestone.

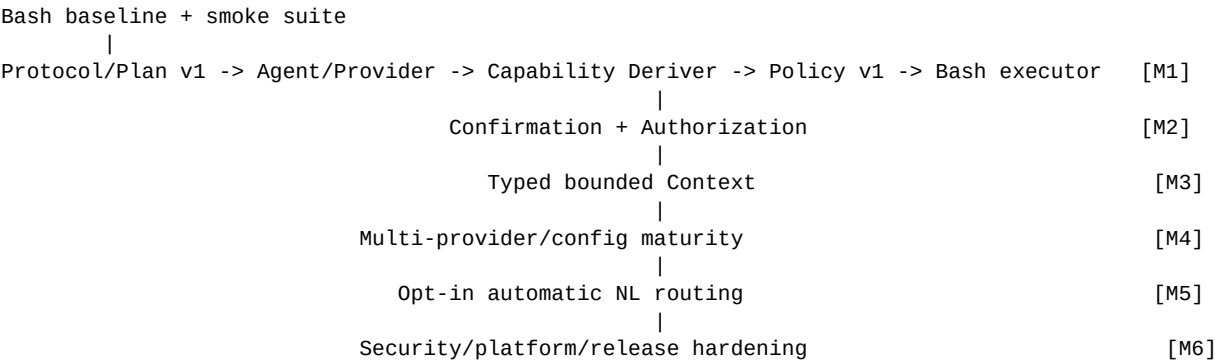
Assumption	Definition
Model assumption	A current frontier coding/reasoning model capable of repository-level edits; exact model may change without changing the milestone contract.
Repository context	Selective diffs/files/symbols are supplied. The full Bash source tree is not repeatedly counted as prompt context.
Excluded	Runtime tokens consumed by hsh users/providers, CI machine time, upstream Bash source size, human legal/security review hours.
Variance	Provider SDK churn, unfamiliar Bash internals, platform bugs and test failures can move actual usage materially outside the range.
Management use	Track acceptance completion, defects and changed LOC/tests separately; do not infer productivity from token burn alone.

Indicative total through M6: approximately 390k-670k assisted-development tokens under the assumptions above. M1 remains approximately 70k-110k after moving foundational decisions into M0 and moving macOS+Linux full support to M6.

12. Ownership and External Dependencies

Area	Accountable role	External dependency / decision
Bash Core / compatibility	Shell Core owner	Pinned GNU Bash 5.3.x baseline; upstream security fixes
IPC / agent lifecycle	Runtime owner	OS local IPC/peer credential primitives; versioned protocol v1
Plan / policy / authorization	Safety/Policy owner	Typed schema; host/filesystem metadata; confirmation contract
Provider adapters / egress	AI Integration owner	Chosen provider APIs/local runtime; credential handling
Context / routing	Agent Experience owner	M3 ContextResult contract; M5 ambiguity contract
Packaging / license	Release owner	GPL compliance, distribution channels, signing/reproducibility
Security validation	Security reviewer	Threat model, adversarial tests, fuzzing; independent review when feasible

12.1 Dependency chain



13. Principal Risks and Explicit Gates

Risk	Failure mode	Roadmap control
Executor split reappears	Rust or plugin bypasses canonical Bash executor/policy	M0 repo rule + M1-AC-08 + architecture test forbidding alternate OS-spawn authority
Wrong target set / free selection expression	Destructive follow-up acts on unintended files/processes	Typed explicit targets only; M3 identity/TTL/job revalidation; no free-form selection language
Policy reload race	Approval under policy A executes under policy B	M2 AuthorizationArtifact binds immutable policy/config snapshot; reload affects later plans
Prompt injection via local output	Prior stdout/file content changes instructions or leaks secrets	M1 local-only output presentation; M3 typed ContextResult + Data-Egress Gate + adversarial tests
IPC peer/session confusion	Another local process injects confirm/execute request	Per-session child, peer verification/session credential, restart epoch, session-bound artifacts
Protocol/parser attack	Malformed provider/IPC input crashes or bypasses validation	Strict decode/default-deny; M1 adversarial tests + parser/schema fuzzing P1
Platform semantic drift	Plan behaves differently on Linux/macOS/WSL	One blocking M1 platform; normalized OS metadata; platform matrix becomes release gate in M6
Provider/config lock-in	Safety semantics depend on vendor	Provider-independent normalized plan/policy; M4 conformance across local+remote adapters
Bash/GPL fork risk	Security divergence or distribution violation	M0 pinned provenance/merge strategy/legal gate; M6 packaging validation

14. Release Gates

Gate	Required before	Evidence
G0 - Provenance/architecture baseline	M1 development	Pinned Bash tag, smoke suite, protocol v1, repo boundary, license checklist
G1 - Direct Bash compatibility	Every milestone tag	Frozen smoke suite + direct-path tests
G2 - No raw eval / canonical executor	Any AI execution	Architecture/integration tests; no Rust alternate exec authority
G3 - Derived capabilities + default-deny policy	M1 AI execution	Policy v1 tests including provider-hint mismatch
G4 - Confirmation + bound authorization	Any M2+ state change	Plan-hash/snapshot/expiry/cancel tests
G5 - Typed context revalidation	Any destructive follow-up	TTL/job/PID/file identity/stale-target tests
G6 - Auto-routing ambiguity safety	M5 auto-routing	Bash precedence + high-risk ambiguity fixtures
G7 - Security/platform/license release review	M6 / 1.0	Fuzz/threat model, platform matrix, audit tests, packaging/provenance validation

15. Immediate Backlog

Priority	Milestone	Backlog item
P0	M0	Pin exact GNU Bash 5.3.x tag/commit and upstream merge policy.
P0	M0	Choose M1 primary platform and freeze named Bash smoke-suite manifest.

Priority	Milestone	Backlog item
P0	M0	Freeze IPC v1 + ExecutionPlan v1 version fields, envelope kinds and error codes.
P0	M0	Record per-session agent lifecycle and provider selection for M1; add deterministic mock provider.
P0	M1	Build/rebrand hsh and preserve direct Bash path.
P0	M1	Implement C bridge + Rust agent handshake/restart epoch.
P0	M1	Implement typed read-only plan subset and local Capability Deriver.
P0	M1	Implement Policy Engine v1 default-deny/read-only rules.
P0	M1	Implement canonical Bash execution adapter and bounded result streaming.
P0	M1	Add end-to-end Portuguese prompt -> plan -> derived policy -> Bash execution -> typed result test.
P1	M1	Add malformed JSON/unknown version/step smuggling/adversarial authorization tests.
P1	M1	Add parser/schema fuzz target to CI on a bounded schedule.
P1	M1	Add non-blocking secondary-platform compile/smoke job.
P1	M1	Document local-only MVP result presentation and provider egress prohibition for raw stdout.

16. MVP Success Statement

M1 succeeds when hsh can be used as Bash on the selected primary platform and an explicit AI request can safely complete a useful read-only system task through the real production boundaries: session IPC, provider normalization, versioned typed plan, local capability derivation, Policy Engine v1, canonical Bash execution and bounded typed results. No manual editing of model output and no hidden alternate executor are permitted.

What M1 proves

M1 proves the architecture and safety boundary, not breadth. M2-M5 expand authority and ergonomics in that order; M6 decides whether the product is ready to ship as 1.0.

Appendix A - v0.1 to v0.2 Corrections

v0.1 issue	v0.2 resolution
Milestone numbering diverged from SRS/UML	Replaced with canonical M0-M6 sequence matching SRS/UML architecture order.
Policy appeared late despite M1 denial requirements	Policy Engine v1 is a named M1 component; M2 expands it to mutation/authorization.
Auto-routing preceded context	Context moved to M3; provider maturity M4; auto-routing moved to M5.
M1 blocked on Linux + macOS	One primary platform is blocking in M1; secondary smoke is non-blocking; full platform gate is M6.
Bash smoke suite undefined	M0 freezes exact Bash baseline and a named smoke-suite manifest.
MVP summary could resend stdout to provider	M1 presentation is local/metadata-based; raw output egress is denied by default.
Schema versioning deferred	IPC and ExecutionPlan v1 are versioned in M0/M1 from first executable implementation.
Developer workflows could bypass capabilities	Developer workflows are cross-cutting and must reuse typed capabilities/policy.
Fuzzing delayed to beta	Protocol/parser adversarial tests and bounded fuzzing begin in M1; M6 expands/release-gates them.
Token budget had no measurement premise	Section 11 defines inclusions, exclusions, model/context assumptions and management limits.
No owners/dependencies	Section 12 adds accountable roles and external dependencies.
Known architecture risks missing	Section 13 adds executor split, wrong-target selection, policy reload and IPC session risks.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_Roadmap.pdf | Source SHA-256: 852d06dd7ad47af3cf1c574d601ee8093c3b6a7deabebe6745b85e2603f79c9b

PATCH PRECEDENCE

This appendix is normative. Where it conflicts with text, examples, tables, milestone labels, repository layouts, acceptance-criterion identifiers, or traceability statements in the v0.2 body, this appendix prevails. Unchanged v0.2 requirements remain in force.

AB-001 - Canonical milestone and acceptance-criterion identifiers

The only milestone identifiers are **M0 through M6**. Acceptance criteria use the global form **M<n>-AC-<nnn>**. Legacy IDs are searchable aliases only and MUST NOT be used in new feature plans, pull requests, ADRs, test names, or release gates.

Legacy source	Canonical mapping rule
SRS AC-001..AC-003	AC-00x -> M0-AC-00x
SRS AC-101..AC-109	AC-1xy -> M1-AC-0xy
SRS AC-201..AC-208	AC-2xy -> M2-AC-0xy
SRS AC-301..AC-307	AC-3xy -> M3-AC-0xy
SRS AC-401..AC-404	AC-4xy -> M4-AC-0xy
SRS AC-501..AC-505	AC-5xy -> M5-AC-0xy
SRS AC-601..AC-605	AC-6xy -> M6-AC-0xy
Roadmap Mx-AC-01..99	Mx-AC-yy -> Mx-AC-1yy (roadmap-specific gate range)
Roadmap M0-AC-06	Alias of M0-AC-003; checklist detail remains normative.
UML component exits	Reference canonical Mx-AC sets; no parallel AC numbering.

Examples: SRS AC-104 becomes **M1-AC-004**; Roadmap M1-AC-07 becomes **M1-AC-107**. They are distinct criteria and no longer collide.

AB-003 - Protocol and ExecutionPlan version

The frozen M0 baseline is **IPC protocol_version = "1.0"** and **ExecutionPlan schema_version = "1.0"**. The SRS example containing **"schema_version": "1.1"** is corrected to **"1.0"**. A later 1.1 bump requires an explicit compatibility rule and ADR.

AB-004 - AuthorizationArtifact exists in M1

Every AI-generated plan that reaches the canonical Bash executor MUST carry a locally issued **AuthorizationArtifact**, including read-only M1 plans. M1 read-only authorization does not require user confirmation when policy allows it; M2 extends the same artifact with confirmation binding for state-changing operations.

The artifact binds at minimum **plan_hash**, **session_id**, **restart_epoch**, **immutable policy/config snapshot hashes**, **uid/gid/cwd**, **effective capabilities**, **issued_at** and **expires_at**. The Bash Compatibility Core SHALL reject AI-plan execution without a valid artifact.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_Roadmap.pdf

AB-002 - Canonical repository layout

The following tree is the single repository baseline. Any alternate tree in the v0.2 body is superseded. No Rust crate may own direct OS process execution for AI-generated plans.

```
hsh/
  bash/                # pinned GNU Bash 5.3.x-derived C core
  bridge/              # C <-> agent IPC integration
  builtins/            # hsh-specific Bash/C builtins
  schemas/
    protocol/
    execution-plan/
  rust/
    hsh-agent/         # per-interactive-session orchestrator
    crates/
      hsh-protocol/
      hsh-intelligence/
      hsh-policy/
      hsh-context/
      hsh-egress/
      hsh-authorization/
      hsh-audit/
      hsh-os/          # metadata/canonicalization only
      hsh-provider/
    providers/
      openai/
      ollama/
  tests/
    compatibility/
    protocol/
    policy/
    adversarial/
    integration/
```

AB-005 - Platform status: product target vs. implementation gate

Product targets: Linux and macOS. **M1 blocking platform:** exactly one of Linux or macOS, selected during M0. The other is smoke/non-blocking through M5 and becomes blocking by M6. WSL remains secondary/documented unless promoted by ADR.

AB-006 - M1 explicit AI surface and Data-Egress Gate

The mandatory M1 surface is **ai "..."**. The **?** shorthand is optional syntax sugar and **MUST NOT** be an M1 release gate. Data-Egress Gate is P0/M1 and runs before every remote-provider transmission.

M1 data class	Remote-provider default
User NL request	Allowed after local redaction/classification
Typed OS metadata required to plan	Allowlisted only
Env vars / credentials / secrets	Denied
File/repository bodies	Denied
stdout / stderr bodies	Denied
Exit status, bounded paths/sizes/counts	Local presentation; egress only if later classified

M1 natural-language-like result presentation MUST be local/deterministic or metadata-only. It SHALL NOT perform a second provider call containing raw stdout/stderr or file bodies.

AB-007 - Schema-known step types vs. milestone authorization

ExecutionPlan v1 MAY define FileOperationStep, ProcessOperationStep, NetworkStep and PrivilegeStep from M0. Schema recognition does not imply authorization. In M1, any plan requiring capabilities outside the read-only M1 set **MUST** be rejected as **unsupported/denied before AuthorizationArtifact issuance**. Write, delete, process control/kill, network side effects and privilege escalation remain unavailable in M1.

Capabilities are always recomputed locally from typed steps, resolved targets, cwd, env_delta, redirections, host profile and filesystem metadata. Provider risk/capability fields are non-authoritative hints.

hsh v0.2.1 - Normative PDF Patch

Applies to: hsh_Roadmap.pdf

AB-008 - UML M6 component exit is normative

The UML component-design traceability section is extended through **M6**. M6 does not add an alternate executor or trust path. Component exit requires:

- compatibility regression against the pinned Bash baseline and documented intentional deviations;
- Linux and macOS release fixtures blocking by 1.0; WSL status documented;
- fuzzing/adversarial coverage for JSON/IPC, typed plan decoding, policy inputs and AuthorizationArtifact verification;
- agent crash/restart isolation that cannot corrupt Bash job-control state;
- audit retention, redaction and integrity controls finalized;
- packaging/license/provenance checks completed; stable v1 protocol/provider contracts frozen;
- no unreviewed raw shell-script execution path in release builds.

AB-009 - Canonical sequence

M0 Foundation -> M1 Explicit AI/read-only + Policy v1 -> M2 Controlled Mutation -> M3 Bounded Context -> M4 Provider/Config Maturity -> M5 Opt-in Automatic NL Routing -> M6 Hardening/1.0.

No milestone may enable a capability assigned to a later milestone merely because its step type already exists in the schema.

AB-010 - Canonical executor and trust path

The only OS process executor for AI-generated plans is the Bash Compatibility Core in C. Rust may plan, normalize, recompute capabilities, evaluate policy, build context, apply data-egress policy, issue authorization metadata and coordinate IPC; Rust SHALL NOT call exec/fork/spawn as an alternate AI-plan execution path.

```
Frontend C / TTY
-> hsh-agent (plan + policy)
-> AuthorizationArtifact
-> Frontend C / confirmation when required
-> Bash Compatibility Core
-> OS fork/exec/job-control/redirection
-> bounded local result
-> optional typed ContextResult (M3+)
```

AB-011 - Roadmap token estimates

Roadmap token ranges are planning heuristics for assisted engineering work only. They are not Codex/OpenAI credits, API runtime budget, a contractual delivery estimate, or an acceptance criterion. Measurement reports SHOULD record model/configuration, input/output mix, whether upstream Bash source was inspected or modified, and excluded generated artifacts.

Migration note

Future v0.3+ editions should fold these rules into the body and remove legacy aliases. Until then, implementation artifacts SHALL cite the canonical IDs and baseline in this appendix.